

MODULE 4

Deep Dive in Python

Topics Covered

1. Python Getting Started & Introduction
2. Variables, Print & Input Functions
3. Data Types & Operators
4. Data Structures: List, Tuple, Dictionary, Sets
5. Conditional & Looping Statements
6. Functions (Overview, Definition, Recursive, Lambda)
7. Higher-Order Functions: Map, Filter
8. File Handling & OS Library
9. Modules, Packages & Important Libraries
10. Exception Handling

Lectures 1 & 2: Python Getting Started & Introduction

1.1 What is Python?

Python is a high-level, interpreted, general-purpose programming language. Created by Guido van Rossum in 1991, it emphasises code readability and simplicity — a beginner can write working code in minutes.

Feature	Details
Creator	Guido van Rossum (1991)
Typing	Dynamically typed
Paradigm	Multi-paradigm: OOP, Functional, Procedural
Interpreter	CPython (default), PyPy, Jython
File extension	.py
Latest series	Python 3.x (Python 2 is deprecated)

1.2 Why Python for AI & Data Science?

- Simple syntax → faster prototyping
- Massive ecosystem: NumPy, Pandas, TensorFlow, PyTorch, Scikit-learn
- Large community & abundant learning resources
- Cross-platform: runs on Windows, macOS, Linux
- Glue language: integrates easily with C/C++, Java, R

□ Real-world analogy

Python is like a Swiss Army knife for programmers — not always the fastest tool, but it has a blade for every job, and you can learn to use it in a day.

1.3 Installing Python

Download from python.org. During installation on Windows, tick 'Add Python to PATH'. Verify installation:

```
# In terminal / command prompt
python --version      # Should show Python 3.x.x
pip --version         # Package installer
```

1.4 Running Python Code

Method	How to Use
Interactive Shell (REPL)	Type python in terminal → line-by-line execution
Script file	Write code in file.py → run with: python file.py
Jupyter Notebook	Browser-based cells; ideal for data science
VS Code / PyCharm	Full IDE with debugging, autocomplete
Google Colab	Cloud-based notebooks; no setup required

1.5 Your First Python Program

```
print("Hello, World!")
```

```
# Output:  
Hello, World!
```

□ Python philosophy

Run: `import this` in the Python shell to see "The Zen of Python" — 19 design principles that guide the language.

Lecture 3: Variables and Print & Input Functions

3.1 Variables

A variable is a named container that stores data. In Python, you don't declare types — you just assign:

```
name = 'Alice'      # string  
age  = 25           # integer  
height = 5.7        # float  
is_student = True   # boolean
```

□ Naming rules

- Start with a letter or underscore (`_`), never a digit
- Case-sensitive: `name` ≠ `Name` ≠ `NAME`

- Use snake_case by convention: student_name, total_marks
- Reserved words (if, for, while, class...) cannot be variable names

3.2 Multiple Assignment

```
x, y, z = 1, 2, 3          # tuple unpacking
a = b = c = 0             # same value to multiple vars
x, y = y, x               # swap without temp variable
```

3.3 The print() Function

print() outputs values to the console. It accepts multiple arguments and several keyword parameters:

```
print("Hello", "World")    # Hello World
print("Hello", "World", sep=", ") # Hello, World
print("Loading", end="...") # Loading... (no newline)
print(f"My name is {name}") # f-string (Python 3.6+)
print("Score: %d" % 95)    # old-style formatting
```

print() Parameter	Purpose
sep=' '	Separator between arguments (default: space)
end='\n'	What to print at the end (default: newline)
file=sys.stdout	Output destination (file, stderr, etc.)
flush=False	Force flush the output buffer

3.4 The input() Function

input() pauses execution, reads a line of text from the user, and returns it as a string:

```
name = input("Enter your name: ")
age = int(input("Enter your age: ")) # cast to int
pi = float(input("Enter pi: "))     # cast to float
print(f"Hello {name}, you are {age} years old.")
```

❏ Common mistake

input() always returns a string. If you need a number, explicitly cast it: int(), float(). Forgetting this causes TypeError when doing arithmetic.

Lecture 4: Data Types

4.1 Built-in Data Types Overview

Category	Types	Example
Text	str	"Hello"
Numeric	int, float, complex	42, 3.14, 2+3j
Boolean	bool	True, False
Sequence	list, tuple, range	[1,2,3], (1,2), range(5)
Mapping	dict	{"a": 1}
Set	set, frozenset	{1, 2, 3}
None	NoneType	None

4.2 Numbers

```
# int - whole numbers (unlimited precision)
x = 1_000_000          # underscores for readability

# float - decimal numbers (IEEE 754 double precision)
pi = 3.14159

# complex - real + imaginary
c = 3 + 4j
print(c.real, c.imag)  # 3.0 4.0

# Type checking
type(x)                # <class 'int'>
isinstance(pi, float)  # True
```

4.3 Strings

```
single = 'Hello'
double = "World"
multi  = """This is
        multi-line"""

# Useful string operations
s = "python"
```

```

print(s.upper())      # PYTHON
print(s.capitalize()) # Python
print(len(s))         # 6
print(s[0])           # p   (indexing)
print(s[-1])          # n   (negative index)
print(s[1:4])         # yth (slicing)

```

4.4 Type Conversion

Function	Converts to
int(x)	Integer (truncates floats, parses numeric strings)
float(x)	Floating-point number
str(x)	String representation
bool(x)	False for 0, None, "", [], {} — True otherwise
list(x)	List from any iterable
tuple(x)	Tuple from any iterable

Lecture 5: Operators in Python

5.1 Arithmetic Operators

Operator	Name	Example
+	Addition	$5 + 3 \rightarrow 8$
-	Subtraction	$5 - 3 \rightarrow 2$
*	Multiplication	$5 * 3 \rightarrow 15$
/	Division	$7 / 2 \rightarrow 3.5$
//	Floor Division	$7 // 2 \rightarrow 3$
%	Modulus	$7 \% 2 \rightarrow 1$
**	Exponentiation	$2 ** 3 \rightarrow 8$

5.2 Comparison Operators

Return True or False:

```
x = 10
```

```

print(x == 10)    # True  - equal
print(x != 5)     # True  - not equal
print(x > 8)      # True  - greater than
print(x < 8)      # False - less than
print(x >= 10)    # True  - greater or equal
print(x <= 10)    # True  - less or equal

```

5.3 Logical Operators

Operator	Behaviour
and	True only if BOTH sides are True
or	True if AT LEAST ONE side is True
not	Reverses the boolean value
<pre> print(True and False) # False print(True or False) # True print(not True) # False # Short-circuit evaluation x = 5 print(x > 0 and x < 10) # True </pre>	

5.4 Assignment & Bitwise Operators

Shorthand	Equivalent
x += 5	x = x + 5
x -= 5	x = x - 5
x *= 2	x = x * 2
x /= 2	x = x / 2
x //= 2	x = x // 2
x **= 2	x = x ** 2
x %= 3	x = x % 3

5.5 Identity & Membership Operators

```

# Identity - checks if two vars point to same object
a = [1, 2]
b = a

```

```
print(a is b)          # True  (same object)
print(a is not b)     # False

# Membership — checks if value is in sequence
fruits = ['apple', 'banana']
print('apple' in fruits) # True
print('mango' not in fruits) # True
```

Lectures 6–8: Lists — Data Structure, Mutability, Slicing, Methods & Comprehension

6.1 What is a List?

A list is an ordered, mutable collection that can hold items of any type, including mixed types.

```
nums      = [1, 2, 3, 4, 5]
mixed     = [42, 'hello', 3.14, True]
nested    = [[1, 2], [3, 4], [5, 6]]
empty     = []

print(len(nums))          # 5
print(nums[0])            # 1  (first element)
print(nums[-1])           # 5  (last element)
print(nested[1][0])       # 3  (nested access)
```

6.2 List Mutability

Unlike strings and tuples, lists can be changed after creation:

```
fruits = ['apple', 'banana', 'cherry']
fruits[1] = 'mango'          # Modify element
print(fruits)                # ['apple', 'mango', 'cherry']

# Lists are passed by reference!
a = [1, 2, 3]
b = a                        # b points to SAME list
b[0] = 99
print(a)                     # [99, 2, 3] — a changed too!

# To copy independently:
b = a.copy()                 # or: b = a[:]
```

☐ Mutable vs Immutable

Mutable: list, dict, set — can be changed in place.

Immutable: str, int, float, tuple — once created, cannot be changed. Any 'change' creates a new object.

6.3 List Slicing

Slicing syntax: `list[start : stop : step]` — creates a new list (does not modify original)

```
nums = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

print(nums[2:6])      # [2, 3, 4, 5]    - index 2 to 5
print(nums[:4])       # [0, 1, 2, 3]   - start to 3
print(nums[6:])       # [6, 7, 8, 9]   - 6 to end
print(nums[::2])      # [0, 2, 4, 6, 8] - every 2nd
print(nums[::-1])     # [9, 8, 7, 6, 5, 4, 3, 2, 1, 0] - reversed
print(nums[1:8:3])    # [1, 4, 7]     - step 3
```

6.4 List Methods

Method	Description & Example
<code>append(x)</code>	Add x to the end: <code>lst.append(5)</code>
<code>insert(i, x)</code>	Insert x at index i: <code>lst.insert(2, 'hi')</code>
<code>extend(iter)</code>	Add all items from iterable: <code>lst.extend([4, 5])</code>
<code>remove(x)</code>	Remove first occurrence of x (ValueError if absent)
<code>pop(i=-1)</code>	Remove & return element at index i (default last)
<code>clear()</code>	Remove all elements
<code>index(x)</code>	Return index of first x
<code>count(x)</code>	Count occurrences of x
<code>sort()</code>	Sort in-place (ascending by default)
<code>sorted(lst)</code>	Return new sorted list (doesn't modify original)
<code>reverse()</code>	Reverse in-place
<code>copy()</code>	Return a shallow copy

6.5 List Comprehension

A concise way to create lists from existing iterables:

```
# Syntax: [expression for item in iterable if condition]
```

```
# Basic - squares
squares = [x**2 for x in range(1, 6)]
# [1, 4, 9, 16, 25]

# With condition - even numbers only
evens = [x for x in range(20) if x % 2 == 0]
# [0, 2, 4, 6, 8, 10, 12, 14, 16, 18]

# String operation
words = ['hello', 'world', 'python']
upper = [w.upper() for w in words]
# ['HELLO', 'WORLD', 'PYTHON']

# Nested comprehension - 3x3 matrix
matrix = [[i*j for j in range(1,4)] for i in range(1,4)]
```

□ Comprehension vs. loop

List comprehensions are typically 35–50% faster than equivalent for-loops because they are optimised at the bytecode level. Prefer them for simple transformations.

Lecture 9: Tuples in Python

9.1 What is a Tuple?

A tuple is an ordered, immutable collection. Once created, its elements cannot be changed, added, or removed.

```
coords = (10, 20)
rgb = (255, 128, 0)
single = (42,) # trailing comma needed for single-element
empty = ()

# Tuples can be created without parentheses (packing)
t = 1, 2, 3
print(type(t)) # <class 'tuple'>
```

9.2 Tuple vs List

Aspect	Tuple	List
Mutability	Immutable	Mutable
Syntax	(1, 2, 3)	[1, 2, 3]

Aspect	Tuple	List
Speed	Faster	Slightly slower
Memory	Less	More
Use case	Fixed data (coords, RGB, DB rows)	Dynamic collections
Hashable?	Yes (can be dict key)	No

9.3 Tuple Unpacking

```

point = (3, 7)
x, y = point           # basic unpacking
print(x, y)            # 3 7

# Extended unpacking with *
first, *rest = (1, 2, 3, 4, 5)
print(first)           # 1
print(rest)            # [2, 3, 4, 5]

# Swap using tuples
a, b = 10, 20
a, b = b, a
print(a, b)            # 20 10

```

9.4 Named Tuples

```

from collections import namedtuple

Point = namedtuple('Point', ['x', 'y'])
p = Point(3, 7)
print(p.x, p.y)        # 3 7
print(p[0])            # 3 (still index-accessible)

```

□ When to use tuples

Use tuples for data that should NOT change: GPS coordinates, RGB colour codes, database records returned from a query, dictionary keys, function return values.

Lecture 10: Dictionary in Data Structure

10.1 What is a Dictionary?

A dictionary is an ordered (Python 3.7+), mutable collection of key-value pairs. Keys must be unique and immutable (strings, numbers, tuples).

```
student = {
    'name' : 'Alice',
    'age' : 22,
    'grade' : 'A'
}

print(student['name'])      # Alice
print(student.get('age'))  # 22
print(student.get('gpa', 0)) # 0 (default if key missing)
```

10.2 CRUD Operations

```
d = {'a': 1, 'b': 2}

# Create / Update
d['c'] = 3          # Add new key
d['a'] = 100        # Update existing key

# Read
print(d.keys())     # dict_keys(['a', 'b', 'c'])
print(d.values())   # dict_values([100, 2, 3])
print(d.items())    # dict_items([('a', 100), ('b', 2), ('c', 3)])

# Delete
del d['b']          # Remove key 'b'
d.pop('c')          # Remove & return value
d.clear()           # Remove all
```

10.3 Iterating Over a Dictionary

```
info = {'name': 'Bob', 'age': 30, 'city': 'Delhi'}

for key in info:
    print(key, '->', info[key])

# Pythonic way
for key, value in info.items():
    print(f'{key}: {value}')
```

10.4 Dictionary Comprehension

```
# Squares as dict
squares = {x: x**2 for x in range(1, 6)}
# {1:1, 2:4, 3:9, 4:16, 5:25}

# Filter only even values
evens = {k: v for k, v in squares.items() if v % 2 == 0}
# {2:4, 4:16}

# Invert a dictionary
inverted = {v: k for k, v in squares.items()}
```

Lecture 11: Sets in Data Structures

11.1 What is a Set?

A set is an unordered, mutable collection of unique elements. Duplicates are automatically removed. Sets are great for membership tests and eliminating duplicates.

```
s = {1, 2, 3, 2, 1}      # duplicates removed automatically
print(s)                 # {1, 2, 3}

# From a list
lst = [1, 2, 2, 3, 3, 4]
unique = set(lst)
print(unique)            # {1, 2, 3, 4}

empty_set = set()       # NOT {} - that creates an empty dict!
```

11.2 Set Operations (Like Maths)

```
A = {1, 2, 3, 4}
B = {3, 4, 5, 6}

print(A | B)    # Union:      {1,2,3,4,5,6}
print(A & B)    # Intersection: {3,4}
print(A - B)    # Difference:  {1,2}
print(A ^ B)    # Symmetric diff: {1,2,5,6}

# Subset / Superset
print({1,2}.issubset(A))    # True
print(A.issuperset({1,2}))  # True
```

11.3 Set Methods

Method	Description
<code>add(x)</code>	Add element x
<code>remove(x)</code>	Remove x (KeyError if not found)
<code>discard(x)</code>	Remove x (no error if not found)
<code>pop()</code>	Remove and return an arbitrary element
<code>clear()</code>	Remove all elements
<code>union(s)</code>	Return union of sets
<code>intersection(s)</code>	Return common elements
<code>difference(s)</code>	Elements in self but not in s
<code>isdisjoint(s)</code>	True if no common elements
Sets are O(1) for lookup	
Checking 'x in my_set' is O(1) average time, vs O(n) for lists. When you need fast membership testing on large data, always prefer a set.	

Lecture 12: Conditional Statements

12.1 if / elif / else

```

score = 78

if score >= 90:
    print('Grade: A')
elif score >= 75:
    print('Grade: B')
elif score >= 60:
    print('Grade: C')
else:
    print('Grade: F')

# Output: Grade: B

```

12.2 Ternary (Conditional) Expression

```
# Syntax: value_if_true if condition else value_if_false
```

```
age = 20
status = 'Adult' if age >= 18 else 'Minor'
print(status)          # Adult

# Nested ternary (keep readable!)
x = 5
label = 'positive' if x > 0 else ('zero' if x == 0 else 'negative')
```

12.3 Truthy & Falsy Values

Falsy (evaluate to False)	Truthy (evaluate to True)
0, 0.0, 0j	Any non-zero number
Empty string ""	Any non-empty string "hello"
None	Any object
Empty list []	Non-empty list [1]
Empty dict {}	Non-empty dict {'a':1}
Empty set set()	Non-empty set {1}
False	True

```
# Pythonic check
lst = []
if not lst:
    print('List is empty')
```

Lectures 13 & 14: Looping Statements

13.1 for Loop

Iterates over any iterable (list, string, range, dict, etc.):

```
# Iterate list
for fruit in ['apple', 'banana', 'cherry']:
    print(fruit)

# Iterate with range
for i in range(5):          # 0,1,2,3,4
    print(i)

for i in range(2, 10, 2):   # 2,4,6,8 (start, stop, step)
    print(i)
```

```
# enumerate - get index & value
for i, val in enumerate(['a', 'b', 'c'], start=1):
    print(i, val)          # 1 a  2 b  3 c

# zip - iterate two lists together
names = ['Alice', 'Bob']
scores = [90, 85]
for name, score in zip(names, scores):
    print(f'{name}: {score}')
```

13.2 while Loop

```
count = 0
while count < 5:
    print(count)
    count += 1

# Infinite loop with break
while True:
    user = input('Enter q to quit: ')
    if user == 'q':
        break
```

13.3 Loop Control Statements

Statement	Effect
break	Exit the loop immediately
continue	Skip the rest of this iteration, go to next
pass	Do nothing — placeholder for empty blocks
else (loop)	Runs ONLY if loop completed without break

```
# continue example - skip odd numbers
for i in range(10):
    if i % 2 != 0:
        continue
    print(i)          # 0 2 4 6 8

# else on loop
for i in range(5):
    if i == 3:
        break
else:
    print('Loop finished without break') # NOT printed
```


Statement	Effect
❑ Loop best practices	
• Use for loops when you know the number of iterations. • Use while loops for condition-based repetition. • Avoid modifying a list while iterating over it — iterate over a copy instead.	

Lectures 15–19: Functions

15.1 What is a Function?

A function is a named, reusable block of code that performs a specific task. Functions promote DRY (Don't Repeat Yourself) coding.

```
# Define
def greet(name):
    '''Returns a greeting string.''' # docstring
    return f'Hello, {name}!'

# Call
message = greet('Alice')
print(message)          # Hello, Alice!
```

15.2 Parameters & Arguments

Type	Example
Positional	<code>greet('Alice')</code>
Keyword	<code>greet(name='Alice')</code>
Default parameter	<code>def greet(name='World'):</code>
*args (var pos)	<code>def total(*nums): sum(nums)</code>
kwargs (var kw)	<code>def show(info): print(info)</code>
<pre># Default parameter def power(base, exp=2): return base ** exp print(power(3)) # 9 (exp defaults to 2) print(power(3, 3)) # 27</pre>	

Type	Example
	<pre># *args - variable positional arguments def add(*nums): return sum(nums) print(add(1, 2, 3, 4)) # 10 # **kwargs - variable keyword arguments def profile(**info): for k, v in info.items(): print(f'{k}: {v}') profile(name='Alice', age=22, city='Mumbai')</pre>

15.3 Return Values

```
# Return multiple values (as tuple)
def min_max(lst):
    return min(lst), max(lst)

lo, hi = min_max([3, 1, 9, 2, 7])
print(lo, hi) # 1 9

# Function with no return → returns None
def say_hi():
    print('Hi!')

result = say_hi() # Hi!
print(result) # None
```

15.4 Recursive Functions

A function that calls itself. Must have a base case to stop recursion:

```
# Factorial
def factorial(n):
    if n == 0 or n == 1: # base case
        return 1
    return n * factorial(n - 1) # recursive case

print(factorial(5)) # 120

# Fibonacci
def fib(n):
    if n <= 1:
        return n
```

```
    return fib(n-1) + fib(n-2)

print(fib(8))    # 21
```

❑ Recursion limit

Python's default recursion limit is 1000. Deep recursion raises `RecursionError`. For large inputs, prefer iterative solutions or increase limit with `sys.setrecursionlimit(n)`.

Lectures 20–22: Lambda, Map & Filter Functions

20.1 Lambda Functions

A lambda is an anonymous (nameless), single-expression function. Syntax: `lambda arguments: expression`

```
# Regular function
def square(x):
    return x ** 2

# Equivalent lambda
square = lambda x: x ** 2
print(square(5))    # 25

# Multi-argument lambda
add = lambda x, y: x + y
print(add(3, 4))    # 7

# Used inline – sorting by second element
pairs = [(1, 'b'), (3, 'a'), (2, 'c')]
pairs.sort(key=lambda p: p[1])
print(pairs)    # [(3, 'a'), (1, 'b'), (2, 'c')]
```

21.1 map() Function

Applies a function to every item in an iterable and returns a map object (lazy iterator):

```
# Syntax: map(function, iterable)

nums = [1, 2, 3, 4, 5]

# Using lambda
squares = list(map(lambda x: x**2, nums))
print(squares)    # [1, 4, 9, 16, 25]
```

```
# Using built-in function
words = ['hello', 'world']
upper = list(map(str.upper, words))
print(upper)           # ['HELLO', 'WORLD']

# map with two iterables
a = [1, 2, 3]
b = [10, 20, 30]
result = list(map(lambda x, y: x + y, a, b))
print(result)          # [11, 22, 33]
```

22.1 filter() Function

Filters elements from an iterable — keeps only those for which the function returns True:

```
# Syntax: filter(function, iterable)

nums = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

# Keep only even numbers
evens = list(filter(lambda x: x % 2 == 0, nums))
print(evens)           # [2, 4, 6, 8, 10]

# Filter non-empty strings
words = ['hello', '', 'world', '', 'python']
non_empty = list(filter(None, words))    # None = identity function
print(non_empty)       # ['hello', 'world', 'python']
```

22.2 map vs filter vs list comprehension

Tool	Purpose	Returns
map(f, iter)	Transform every element	map object (lazy)
filter(f, iter)	Keep elements where f is True	filter object (lazy)
[... for ...]	Both transform & filter	list (eager)

❑ Prefer comprehensions for readability

`[x**2 for x in nums if x%2==0]` is often more readable than combining map and filter. Use map/filter when passing to APIs that expect iterables, or when working with very large datasets (lazy evaluation saves memory).

Lectures 23 & 24: File Handling

23.1 Opening & Closing Files

```
# Always use 'with' - auto-closes even on exceptions
with open('data.txt', 'r') as f:
    content = f.read()
    print(content)
```

Mode	Description
'r'	Read (default) — error if file doesn't exist
'w'	Write — creates file; overwrites existing
'a'	Append — adds to end; creates if not exists
'x'	Create — error if file already exists
'rb'	Read binary (images, PDFs, etc.)
'wb'	Write binary
'r+'	Read & write (file must exist)

23.2 Reading Files

```
with open('sample.txt', 'r') as f:

    # Read entire file as one string
    content = f.read()

    # Read into list of lines
    f.seek(0)          # reset to start
    lines = f.readlines()

    # Read line by line (memory-efficient for large files)
    f.seek(0)
    for line in f:
        print(line.strip())    # strip() removes \n
```

23.3 Writing & Appending

```
# Write (overwrites file if it exists)
with open('output.txt', 'w') as f:
    f.write('Hello, File!\n')
    f.writelines(['Line 1\n', 'Line 2\n'])
```

```
# Append (adds to end)
with open('output.txt', 'a') as f:
    f.write('New line added\n')
```

23.4 Working with CSV & JSON

```
import csv, json

# CSV read
with open('data.csv') as f:
    reader = csv.DictReader(f)
    for row in reader:
        print(row['name'], row['score'])

# JSON read / write
data = {'name': 'Alice', 'scores': [90, 85, 92]}

with open('data.json', 'w') as f:
    json.dump(data, f, indent=4)

with open('data.json') as f:
    loaded = json.load(f)
    print(loaded['name'])      # Alice
```

Lecture 25: OS Library

25.1 What is the os Module?

The `os` module provides a way to interact with the operating system — navigate directories, manage files and folders, read environment variables, etc.

```
import os

print(os.getcwd())          # Current working directory
os.chdir('/path/to/folder') # Change directory
print(os.listdir('.'))      # List files in current dir

# Create / remove directories
os.mkdir('new_folder')      # Create single directory
os.makedirs('a/b/c')        # Create nested directories
os.rmdir('new_folder')      # Remove empty directory

# File operations
```

```
os.rename('old.txt', 'new.txt')
os.remove('file.txt')
print(os.path.exists('file.txt')) # True / False
print(os.path.getsize('data.csv')) # Size in bytes
```

25.2 os.path — Path Utilities

Function	Description
os.path.join(a, b)	Join paths safely (handles OS separators)
os.path.exists(p)	Check if path exists
os.path.isfile(p)	True if p is a file
os.path.isdir(p)	True if p is a directory
os.path.basename(p)	File name from full path
os.path.dirname(p)	Directory portion of path
os.path.splitext(p)	Split name and extension: ('file', '.txt')
os.path.abspath(p)	Absolute path

```
import os

path = '/home/user/documents/notes.txt'
print(os.path.basename(path)) # notes.txt
print(os.path.dirname(path)) # /home/user/documents
print(os.path.splitext(path)) # ('/home/user/documents/notes', '.txt')

# Safe cross-platform path building
full = os.path.join('folder', 'sub', 'file.txt')
# Windows: folder\sub\file.txt / Linux: folder/sub/file.txt
```

Lectures 26 & 27: Modules, Packages & Important Libraries

26.1 What is a Module?

A module is any Python file (.py). You can import it to reuse its code.

```
# mymath.py
def add(a, b):
    return a + b
```

```
PI = 3.14159

# main.py
import mymath
print(mymath.add(2, 3))      # 5
print(mymath.PI)            # 3.14159

# Import specific items
from mymath import add, PI
print(add(10, 5))           # 15

# Alias
import numpy as np
```

26.2 Packages

A package is a directory containing an `__init__.py` file and multiple modules. This lets you organise related modules under a namespace.

```
mypackage/
  __init__.py
  math_utils.py
  string_utils.py

# Usage
from mypackage import math_utils
from mypackage.string_utils import capitalize_all
```

27.1 Important Standard Library Modules

Module	Key Uses
math	sqrt, floor, ceil, pi, factorial, log
random	random(), randint(), choice(), shuffle(), sample()
datetime	date, time, datetime, timedelta — dates & times
os	File system, environment variables
sys	argv, path, exit(), version
re	Regular expressions — search, match, findall, sub
json	load, dump — read/write JSON
collections	defaultdict, Counter, deque, OrderedDict, namedtuple
itertools	product, combinations, permutations, chain
functools	reduce, partial, lru_cache (memoization)

27.2 Essential Third-Party Libraries (for AI/ML)

Library	Purpose
NumPy	Numerical computing — arrays, linear algebra, broadcasting
Pandas	Data manipulation — DataFrames, CSV/Excel, groupby, merge
Matplotlib	2D plotting — line, bar, scatter, histogram, heatmap
Seaborn	Statistical visualisation built on Matplotlib
Scikit-learn	ML algorithms, preprocessing, evaluation metrics
TensorFlow	Deep learning framework by Google
PyTorch	Deep learning framework by Meta (used in research)
OpenCV	Computer vision — images, video processing
NLTK / spaCy	Natural language processing

```
# Quick NumPy example
import numpy as np

a = np.array([1, 2, 3, 4, 5])
print(a.mean())           # 3.0
print(a * 2)              # [2 4 6 8 10]

# Quick Pandas example
import pandas as pd

df = pd.read_csv('data.csv')
print(df.head())          # First 5 rows
print(df.describe())      # Statistical summary
```

Lecture 28: Exception Handling

28.1 What is an Exception?

An exception is a runtime error that interrupts normal program flow. Python raises exceptions for invalid operations. Without handling them, the program crashes.

Common Exception	When it Occurs
ZeroDivisionError	Division by zero
ValueError	Invalid value for an operation
TypeError	Wrong data type

Common Exception	When it Occurs
IndexError	List index out of range
KeyError	Dictionary key not found
FileNotFoundError	File doesn't exist
AttributeError	Object has no such attribute
ImportError	Module not found
NameError	Variable not defined
RecursionError	Maximum recursion depth exceeded

28.2 try / except / else / finally

```
try:
    num = int(input('Enter a number: '))
    result = 10 / num
except ZeroDivisionError:
    print('Cannot divide by zero!')
except ValueError:
    print('Please enter a valid integer.')
except Exception as e:
    # Catch-all
    print(f'Unexpected error: {e}')
else:
    print(f'Result: {result}') # Runs if NO exception occurred
finally:
    print('Execution complete') # ALWAYS runs
```

28.3 Raising Exceptions

```
def check_age(age):
    if age < 0:
        raise ValueError(f'Age cannot be negative: {age}')
    if age > 150:
        raise ValueError(f'Age seems unrealistic: {age}')
    return f'Age {age} is valid'

try:
    print(check_age(-5))
except ValueError as e:
    print(e) # Age cannot be negative: -5
```

28.4 Custom Exceptions

```
# Define a custom exception
class InsufficientFundsError(Exception):
    def __init__(self, balance, amount):
        super().__init__(f'Balance {balance} is less than {amount}')
        self.balance = balance
        self.amount = amount

# Use it
def withdraw(balance, amount):
    if amount > balance:
        raise InsufficientFundsError(balance, amount)
    return balance - amount

try:
    withdraw(100, 500)
except InsufficientFundsError as e:
    print(e)    # Balance 100 is less than 500
```

□ Best practice

- Always catch specific exceptions, not bare 'except:' — it hides bugs.
- Use 'finally' to release resources (close files, DB connections).
- Prefer the 'with' statement (context managers) over manual try/finally for files.

End of Module 4 Notes — Generative AI Course